Gurobi model runs well

```
Gurobi Optimizer version 13.0.1 build v13.0.1rc0 (linux64gpu - "Rocky Linux 9.7 (Blue Onyx)")

CPU model: Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz, instruction set [SSE2|AVX|AVX2|AVX512]
Thread count: 40 physical cores, 80 logical processors, using up to 32 threads

Optimize a model with 395 rows, 349 columns and 984 nonzeros (Min)
Model fingerprint: 0x969be3c9
Model has 41 linear objective coefficients and an objective constant of 1.3637664712000002e+04
Model has 861 quadratic objective terms
Model has 147 SOS constraints
Variable types: 200 continuous, 149 integer (147 binary)
Coefficient statistics:
  Matrix range     [1e+00, 5e+01]
  Objective range  [4e-01, 3e+05]
  QObjective range [5e-03, 4e+06]
  Bounds range     [1e+00, 1e+00]
  RHS range        [1e+00, 1e+02]

Presolve removed 31 rows and 26 columns
Presolve time: 0.01s
Presolved: 364 rows, 323 columns, 1186 nonzeros
Presolved model has 139 SOS constraint(s)
Presolved model has 861 quadratic objective terms
Variable types: 182 continuous, 141 integer (139 binary)
Found heuristic solution: objective 4890.1372759
Found heuristic solution: objective 4889.4985804

Root relaxation: objective 2.643612e+03, 385 iterations, 0.00 seconds (0.00 work units)

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0 2643.61159    0   39 4889.49858 2643.61159  45.9%     -    0s
H    0     0                    3363.4018068 2643.61159  21.4%     -    0s
     0     0 2643.61159    0   39 3363.40181 2643.61159  21.4%     -    0s
H    0     0                    3360.9530393 2643.61159  21.3%     -    0s
H    0     0                    3360.4879357 2643.61159  21.3%     -    0s
H    0     0                    3359.5289018 2643.61159  21.3%     -    0s
     0     2 2643.61159    0   39 3359.52890 2643.61159  21.3%     -    1s
*   26     8               6    2854.3324138 2697.71582  5.49%  14.9    1s

Explored 42 nodes (900 simplex iterations) in 1.46 seconds (0.05 work units)
Thread count was 32 (of 80 available processors)

Solution count 6: 2854.33 3359.53 3360.95 ... 4890.14

Optimal solution found (tolerance 1.00e-04)
Best objective 2.854332413821e+03, best bound 2.854332413821e+03, gap 0.0000%
```

The gurobi runs correctly, and the license is now accessed.
Just need to run some code in Jupiter Terminal before running the model.

Validation

Paper:

introduce the true sparsity into the model that allows for $J > N$. With these simulated three-group models, we applied the new fusion-adapted L_0 constraint method, using GUROBI to fit models with $K = 2, 3, 4$, and used BIC to select the final model with the best goodness of fit among the candidate in order to determine the method's sensitivity in selecting the right-sized model. BIC was chosen as the goodness-of-fit (GoF) criterion, as it is deemed best suited for discovery in connection to the goal of uncovering the true signals for the underlying model. In many occasions, BIC has been theoretically proved to be selection consistent for the true underlying data generating process. Other measures, such as AIC, are regarded as being more suited for the evaluation of prediction, which is not aligned with the scope of this analytic interest. (Kass and Raftery (1995), Vrieze (2012)). Additionally, we conducted simulation experiments for the $K^* = 4$-group model with corresponding true cutoffs $(c_1^*, c_2^*, c_3^*, c_4^*)$ and true effect sizes $(\beta_1^*, \beta_2^*, \beta_3^*, \beta_4^*)$ (refer to the Supplementary Material (Banker, Zhang and Song (2024))).

Validation of the model result in the paper starts by selecting the number of segments (K) using BIC (by comparing (K = 2, 3, 4)).

Result:

```
===== BIC comparison =====
   K  status          rss          bic  gamma     beta1     beta2  jump1  \
0  4       2  2715.076026  1031.965470    0.0  0.208229 -0.439594    1.0
1  3       2  2854.332414  1058.800768    0.0  0.158086 -0.196386    1.0
2  2       2  3363.401807  1177.369964    0.0  0.085786 -0.009800    1.0

      beta3  jump2     beta4  jump3
0  0.309741    2.0 -0.000317    3.0
1  0.016750    2.0       NaN    NaN
2       NaN    NaN       NaN    NaN
```

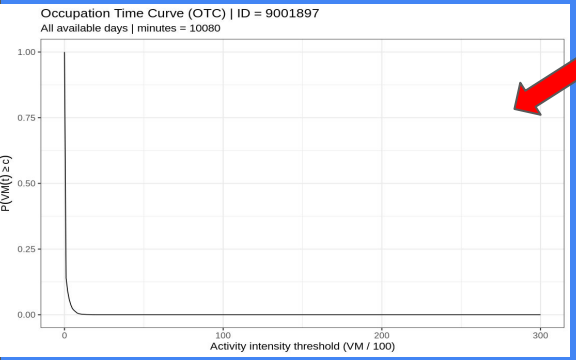
In my results, (K = 4) gives the smallest BIC, which differs from what is suggested in the paper.
Also, beta_4 is so small, it is doubtable that K=4 is valid.

AUC establishment check

```
# A tibble: 6 × 62
      ID n_minutes AUC_1 AUC_2  AUC_3  AUC_4   AUC_5  AUC_6  AUC_7   AUC_8 AUC_9 AUC_10 AUC_11 AUC_12 AUC_13
   <int>     <int> <dbl> <dbl>  <dbl>  <dbl>   <dbl>  <dbl>  <dbl>   <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
1 9000099     10080  59.4 37.8  28.5   24.0    21.5   20.7   20.6    19.3  7.19   1.34   1.24   1.04  0.942
2 9001695     10080  73.6 39.4  23.8   14.5    10.1   7.59   5.26    3.27  2.48   1.64   1.04  0.744  0.546
3 9001897     10080  11.9  1.49  0.347  0       0      0      0       0     0      0      0      0      0
4 9002116     12960  35.0 14.9   6.21   3.16    1.97   0.926  0.617   0.270 0.231  0.0772 0      0      0
5 9002817     10080  40.0 17.8  10.3    7.94    4.81   2.28   0.744   0.0992 0     0      0      0      0
6 9003126     10080  64.6 22.9   5.36   0.595   0.0496 0      0       0     0      0      0      0      0
# ℹ 47 more variables: AUC_14 <dbl>, AUC_15 <dbl>, AUC_16 <dbl>, AUC_17 <dbl>, AUC_18 <dbl>, AUC_19 <dbl>,
#   AUC_20 <dbl>, AUC_21 <dbl>, AUC_22 <dbl>, AUC_23 <dbl>, AUC_24 <dbl>, AUC_25 <dbl>, AUC_26 <dbl>,
#   AUC_27 <dbl>, AUC_28 <dbl>, AUC_29 <dbl>, AUC_30 <dbl>, AUC_31 <dbl>, AUC_32 <dbl>, AUC_33 <dbl>,
#   AUC_34 <dbl>, AUC_35 <dbl>, AUC_36 <dbl>, AUC_37 <dbl>, AUC_38 <dbl>, AUC_39 <dbl>, AUC_40 <dbl>,
#   AUC_41 <dbl>, AUC_42 <dbl>, AUC_43 <dbl>, AUC_44 <dbl>, AUC_45 <dbl>, AUC_46 <dbl>, AUC_47 <dbl>,
#   AUC_48 <dbl>, AUC_49 <dbl>, AUC_50 <dbl>, AUC_51 <dbl>, AUC_52 <dbl>, AUC_53 <dbl>, AUC_54 <dbl>,
#   AUC_55 <dbl>, AUC_56 <dbl>, AUC_57 <dbl>, AUC_58 <dbl>, AUC_59 <dbl>, AUC_60 <dbl>
```

1

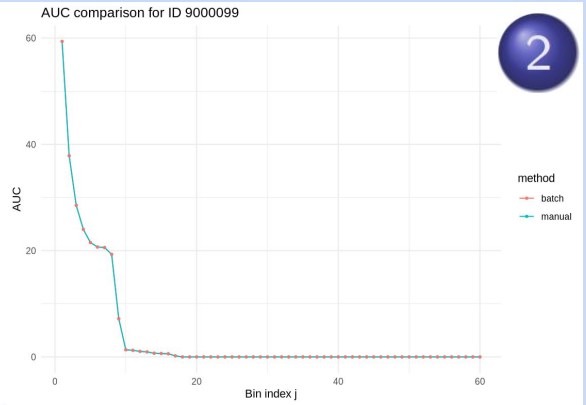
There are some AUC values seem strange, we apply two ways of checking:



First, we look at the OTC of these IDs, and after that, we can see the zeros for later AUC cutoffs are reasonable or not.

Second, we compare AUCs of a direct (manual) computation for a single subject and a vectorized (batch) computation applied across all subjects.

```
> print(head(check_batch, 20))
    j AUC_manual  AUC_batch diff
1   1 59.3750000 59.3750000    0
2   2 37.8472222 37.8472222    0
3   3 28.5218254 28.5218254    0
4   4 24.0079365 24.0079365    0
5   5 21.5277778 21.5277778    0
6   6 20.6845238 20.6845238    0
7   7 20.5853175 20.5853175    0
8   8 19.2956349 19.2956349    0
9   9  7.1924603  7.1924603    0
10 10  1.3392857  1.3392857    0
11 11  1.2400794  1.2400794    0
12 12  1.0416667  1.0416667    0
13 13  0.9424603  0.9424603    0
14 14  0.6944444  0.6944444    0
15 15  0.6448413  0.6448413    0
16 16  0.5952381  0.5952381    0
17 17  0.1984127  0.1984127    0
18 18  0.0000000  0.0000000    0
19 19  0.0000000  0.0000000    0
20 20  0.0000000  0.0000000    0
```



2

After checking, the AUCs seems correct.

Typical exploratory data analysis (EDA) for high-frequency accelerometer data would include examining the distribution of activity intensity, normalizing for different observation durations, and identifying patterns across subjects.
However, these steps are inherently addressed by the OTC and its derived AUC features, which summarize the distribution of activity levels, normalize by total time, and provide a structured representation suitable for modeling.

Concern of Jump (c and jump)

	$J = 300$						$J = 60$								
	$K = 2$			$K = 3$			$K = 2$			$K = 3$			$K = 4$		
Parameters	Est	p-val	perm.	Est	p-val	perm.	Est	p-val	perm.	Est	p-val	perm.	Est	p-val	perm.
β_1	19.50	0.03	0.04	4.17	0.01	0.01	4.73	0.01	0.02	4.21	0.01	0.01	0.13	0.53	0.52
β_2	−0.07	0.53	0.54	−0.36	0.02	0.02	−0.12	0.34	0.34	−0.37	0.02	0.02	−126.26	<0.01	<0.01
β_3	–	–	–	13.00	0.01	0.02	–	–	–	8.56	0.01	0.02	62.91	<0.01	<0.01
β_4	–	–	–	–	–	–	–	–	–	–	–	–	−0.54	0.70	0.70
c_1	3	–	–	20	–	–	15	–	–	20	–	–	240	–	–
c_2	–	–	–	293	–	–	–	–	–	290	–	–	245	–	–
c_3	–	–	–	–	–	–	–	–	–	–	–	–	255	–	–

After that, I also look at the jumps (denoted as c_1, c_2 or c_3) from the paper. For example, like for J=300 and K=3 case, c_1=20 and c_2=293, which means that the first cutoff is between AUC_1 and AUC_20; the second is between AUC_21 and AUC_293; the final cutoff is between AUC_294 and AUC_500.

```
===== BIC comparison =====
   K  status          rss          bic  gamma     beta1     beta2  jump1  \
0  4       2  2715.076026  1031.965470    0.0  0.208229 -0.439594    1.0
1  3       2  2854.332414  1058.800768    0.0  0.158086 -0.196386    1.0
2  2       2  3363.401807  1177.369964    0.0  0.085786 -0.009800    1.0

      beta3  jump2     beta4  jump3
0  0.309741    2.0 -0.000317    3.0
1  0.016750    2.0       NaN    NaN
2       NaN    NaN       NaN    NaN
```

The cutpoints obtained under K=4 appear highly unbalanced, with one large interval and several extremely narrow ones.

This behavior is expected under overfitting, especially given the high multicollinearity among AUC features since K=4 is not final model.

However, my jumps seem to be quite weird.

Summary:

The data preparation has been carefully validated.

In particular, the AUC features derived from the OTC were verified through both direct (manual) computation and batch implementation, confirming their correctness.

However, the **estimated cutpoints (jumps)** appear irregular and inconsistent with the results reported in the paper.

This raises concerns about potential issues in the implementation rather than the data itself.

Therefore, the **next step is to systematically cross-check the current code against the original methodology in the paper**, examining each component to identify possible discrepancies in the implementation.